run-llama / llama_index Public

Code | Issues 176 | Pull requests 124 | Discussions | Actions

240 Branches 465 Tags Go to file Go to file Code

NIK-TIGER-BILL fix: JSON-serialize ToolCallBlock.tool_kwargs in to_open...

9ce3f29 · 2 days ago

.github	docs: fix typos and broken an...	3 weeks ago
docs	docs: fix broken OpenSearch ...	last week
llama-dev	Delete old folders (#21286)	27 days ago
llama-index-core	fix: avoid mutating response.r...	last week
llama-index-instrument...	remaining cleanup, uv lock bu...	last month
llama-index-integrations	fix: JSON-serialize ToolCallBlo...	2 days ago
llama-index-utils	chore(deps): bump the pip gro...	last week
scripts	fix examples rsync (#20999)	last month
.gitignore	remove duplicate docs config...	last month
.pre-commit-config.yaml	[pre-commit] Stop sorting uv.l...	last week
.readthedocs.yaml	docs: pull Workflows documen...	8 months ago
CHANGELOG.md	Release 0.14.21 (#21427)	last week
CITATION.cff	`prettier` in `pre-commit`: forma...	3 years ago
CODE_OF_CONDUCT.md	Create CODE_OF_CONDUCT....	2 years ago
CONTRIBUTING.md	docs: fix typos and broken an...	3 weeks ago
LICENSE	Updated `ruff` and `Makefile`, a...	3 years ago
Makefile	add async to memory and pos...	last year
README.md	Delete old folders (#21286)	27 days ago
RELEASE_HEAD.md	Title for Voyage Multimodal no...	11 months ago

SECURITY.md	deprecate some packages (#2...	28 days ago
STALE.md	Policy for removing stale pack...	2 years ago
docs.config.mjs	remove duplicate docs config...	last month
pyproject.toml	Release 0.14.21 (#21427)	last week
uv.lock	feat(llms-openai): add gpt-5.5...	3 days ago

README | Code of conduct | Contributing | MIT license | Security

🗂️ LlamaIndex 🦙

downloads 10M/month | Build Package passing | contributors 473 | chat 1.6k online | Follow @llama_index | r/LlamaIndex 5k | Phorm Ask AI

LlamaIndex OSS (by LlamaIndex) is an open-source framework to build agentic applications. **Parse** is our enterprise platform for agentic OCR, parsing, extraction, indexing and more. You can use LlamaParse with this framework or on its own; see LlamaParse below for signup and product links.

> 📚 **Documentation:**
>
> - LlamaParse
> - LlamaIndex OSS
> - LlamaAgents

Building with LlamaIndex typically involves working with LlamaIndex core and a chosen set of integrations (or plugins). There are two ways to start building with LlamaIndex in Python:

1. **Starter**: `llama-index`. A starter Python package that includes core LlamaIndex as well as a selection of integrations.

2. **Customized**: `llama-index-core`. Install core LlamaIndex and add your chosen LlamaIndex integration packages on LlamaHub that are required for your application. There are over 300 LlamaIndex integration packages that work seamlessly with core, allowing you to build with your preferred LLM, embedding, and vector store providers.

The LlamaIndex Python library is namespaced such that import statements which include `core` imply that the core package is being used. In contrast, those statements without `core` imply that an integration package is being used.

```
# typical pattern
from llama_index.core.xxx import ClassABC  # core submodule xxx
```

```
from llama_index.xxx.yyy import (
    SubclassABC,
)  # integration yyy for submodule xxx

# concrete example
from llama_index.core.llms import LLM
from llama_index.llms.openai import OpenAI
```

LlamaParse (document agent platform)

LlamaParse is its own platform—focused on document agents and agentic OCR. It includes **Parse** (parsing), **LlamaAgents** (deployed document agents), **Extract** (structured extraction), and **Index** (ingest and RAG). You can use it with the LlamaIndex framework or standalone.

- Sign up for LlamaParse — Create an account and get your API key.
- **Parse** — Agentic OCR and document parsing (130+ formats). Docs
- **Extract** — Structured data extraction from documents. Docs
- **Index** — Ingest, index, and RAG pipelines. Docs
- **Split** — Split large documents into subcategories. Docs
- **Agents** — Build end-to-end document agents with `Workflows` and Agent Builder. Docs

Important Links

Documentation

X (formerly Twitter)

LinkedIn

Reddit

Discord

🚀 Overview

NOTE: This README is not updated as frequently as the documentation. Please check out the documentation above for the latest updates!

Context

- LLMs are a phenomenal piece of technology for knowledge generation and reasoning. They are pre-trained on large amounts of publicly available data.
- How do we best augment LLMs with our own private data?

We need a comprehensive toolkit to help perform this data augmentation for LLMs.

Proposed Solution

That's where **LlamaIndex** comes in. LlamaIndex is a "data framework" to help you build LLM apps. It provides the following tools:

- Offers **data connectors** to ingest your existing data sources and data formats (APIs, PDFs, docs, SQL, etc.).
- Provides ways to **structure your data** (indices, graphs) so that this data can be easily used with LLMs.
- Provides an **advanced retrieval/query interface over your data**: Feed in any LLM input prompt, get back retrieved context and knowledge-augmented output.
- Allows easy integrations with your outer application framework (e.g. with LangChain, Flask, Docker, ChatGPT, or anything else).

LlamaIndex provides tools for both beginner users and advanced users. Our high-level API allows beginner users to use LlamaIndex to ingest and query their data in 5 lines of code. Our lower-level APIs allow advanced users to customize and extend any module (data connectors, indices, retrievers, query engines, reranking modules), to fit their needs.

💡 Contributing

Interested in contributing? Contributions to LlamaIndex core as well as contributing integrations that build on the core are both accepted and highly encouraged! See our Contribution Guide for more details.

New integrations should meaningfully integrate with existing LlamaIndex framework components. At the discretion of LlamaIndex maintainers, some integrations may be declined.

📄 Documentation

Full documentation can be found here

Please check it out for the most up-to-date tutorials, how-to guides, references, and other resources!

💻 Example Usage

```
# custom selection of integrations to work with core
pip install llama-index-core
pip install llama-index-llms-openai
pip install llama-index-llms-ollama
pip install llama-index-embeddings-huggingface
```

Examples are in the `docs/examples` folder. Indices are in the `indices` folder (see list of indices below).

To build a simple vector store index using OpenAI:

```
import os

os.environ["OPENAI_API_KEY"] = "YOUR_OPENAI_API_KEY"

from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader("YOUR_DATA_DIRECTORY").load_data()
index = VectorStoreIndex.from_documents(documents)
```

To build a simple vector store index using non-OpenAI LLMs, e.g. LLMs hosted through Ollama:

```
from llama_index.core import Settings, VectorStoreIndex, SimpleDirectoryReader
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llama_index.llms.ollama import Ollama
from transformers import AutoTokenizer

# set the LLM
Settings.llm = Ollama(
    model="llama-3.1:latest",
    request_timeout=360.0,
)

# set tokenizer to match LLM
Settings.tokenizer = AutoTokenizer.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct"
)

# set the embed model
Settings.embed_model = HuggingFaceEmbedding(
    model_name="BAAI/bge-small-en-v1.5"
)

documents = SimpleDirectoryReader("YOUR_DATA_DIRECTORY").load_data()
index = VectorStoreIndex.from_documents(
    documents,
)
```

To query:

```
query_engine = index.as_query_engine()
query_engine.query("YOUR_QUESTION")
```

By default, data is stored in-memory. To persist to disk (under `./storage`):

```
index.storage_context.persist()
```

To reload from disk:

```
from llama_index.core import StorageContext, load_index_from_storage

# rebuild storage context
storage_context = StorageContext.from_defaults(persist_dir="./storage")
# load index
index = load_index_from_storage(storage_context)
```

A note on Verification of Build Assets

By default, `llama-index-core` includes a `_static` folder that contains the nltk and tiktoken cache that is included with the package installation. This ensures that you can easily run `llama-index` in environments with restrictive disk access permissions at runtime.

To verify that these files are safe and valid, we use the github `attest-build-provenance` action. This action will verify that the files in the `_static` folder are the same as the files in the `llama-index-core/llama_index/core/_static` folder.

To verify this, you can run the following script (pointing to your installed package):

```
#!/bin/bash
STATIC_DIR="venv/lib/python3.13/site-packages/llama_index/core/_static"
REPO="run-llama/llama_index"

find "$STATIC_DIR" -type f | while read -r file; do
    echo "Verifying: $file"
    gh attestation verify "$file" -R "$REPO" || echo "Failed to verify: $file"
done
```

📖 Citation

Reference to cite if you use LlamaIndex in a paper:

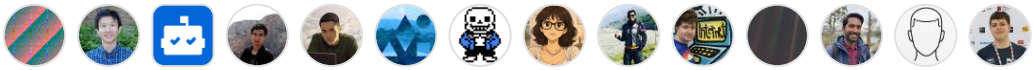
```
@software{Liu_LlamaIndex_2022,
author = {Liu, Jerry},
doi = {10.5281/zenodo.1234},
month = {11},
title = {{LlamaIndex}},
url = {https://github.com/jerryjliu/llama_index},
year = {2022}
}
```

Releases 493

v0.14.21 Latest
last week

+ 492 releases

Contributors 1,865



+ 1,851 contributors

Languages

- Python 71.9%
- Jupyter Notebook 25.5%
- Makefile 1.9%
- JavaScript 0.4%
- HTML 0.3%
- Shell 0.0%